PedagogyEngine AI

An Interactive, Personalized, Multilingual AI Teacher

Project Documentation

Version 1.0.0

Built on Streamlit · Groq / Gemini LLMs · LangChain · ChromaDB · Edge TTS · FFmpeg

Table of Contents

1. Problem Statement

Traditional self-study is a one-size-fits-all experience. A learner working through a textbook, a lecture PDF, or a slide deck gets the same explanation, the same pace, and the same language regardless of whether they are a complete beginner or already have a strong foundation — and regardless of whether they have five minutes or an hour to spend.

This creates several concrete gaps that PedagogyEngine AI sets out to close:

- **No adaptive pacing or difficulty:** Static material cannot simplify itself when a learner struggles, or accelerate and go deeper when a learner is clearly ready to move on.
- **No diagnosis of misunderstanding:** A wrong answer on a quiz rarely explains why it was wrong — is it a calculation slip, a terminology mix-up, or a genuinely missing concept? Without that diagnosis, remediation is generic and often ineffective.
- **No grounding in the learner's own material:** Generic AI chatbots answer from general knowledge, not from the specific textbook, lecture notes, or slide deck the learner has been given, which can produce answers that don't match what will actually be examined or used.
- **No language flexibility:** Learners who think in Hindi, Tamil, or a Hindi-English mix (Hinglish) are often forced to learn technical subjects purely in English, which adds a translation burden on top of the subject itself.
- **No persistence of progress:** Each study session starts from zero — there is no memory of which concepts were already mastered, which were shaky, or what the learner's preferred teaching style is.
- **No engaging, teacher-like delivery:** Reading a PDF is passive. A real tutor explains, demonstrates, asks questions, watches how the learner responds, and adjusts — a loop that static content simply cannot replicate.

The result is a widening gap between the promise of personalized, one-on-one tutoring — historically available only to a privileged few — and the reality of what most self-learners actually get.

2. Solution Overview

PedagogyEngine AI is an interactive, AI-powered teacher built as a Streamlit web application. A learner uploads their own study material (PDF, DOCX, PPTX, or TXT), sets up a short personalization profile, and is then taught by an AI "teacher" that behaves like a human tutor rather than a chatbot — following a genuine teaching loop of introduce → explain → demonstrate → question → evaluate → adapt → continue.

The system is deliberately architected as a set of independent, single-responsibility engines rather than one large prompt. A dedicated decision-making agent (the Teacher Agent) plans each next pedagogical action; a Retrieval-Augmented Generation (RAG) engine grounds every explanation in the learner's own uploaded material; an Assessment Engine grades responses and computes mastery; a Misconception Engine diagnoses why an answer was wrong and generates targeted remediation; and dedicated Audio, Visual, Avatar and Video engines turn the lesson into narrated, illustrated, multilingual audio-visual content — all wrapped in a persistent learner profile that remembers progress across sessions.

The result is a tutor that:

- teaches only from what the learner actually uploaded (with source citations), not generic internet knowledge;
- adapts difficulty, pacing, and teaching style to the individual learner in real time;
- diagnoses the specific misconception behind a wrong answer and remediates it directly, rather than simply repeating the same explanation;
- teaches in the learner's language of choice — English, Hindi, Tamil, Spanish, or Hinglish;
- speaks the lesson aloud with natural neural text-to-speech and an animated on-screen avatar;
- can render a complete narrated lesson as a downloadable MP4 video;
- remembers every learner's mastery per topic and per concept across sessions, and recommends what to study next.

3. Key Features

Feature	Description
Document-grounded teaching	Upload PDF / DOCX / PPTX / TXT material; the tutor teaches strictly from retrieved, cited chunks of that material via RAG.
Human-like teaching loop	A dedicated Teacher Agent decides the next pedagogical action (introduce, explain, demonstrate, ask, evaluate, simplify, remediate, increase difficulty, recap, continue) instead of just generating text.
Deep personalization	Learner level, learning goal, teaching style, explanation depth, session duration, and language are all configurable and shape every generated lesson.
Adaptive mastery tracking	Per-concept and per-topic mastery scores are calculated from assessment performance and persisted, driving difficulty and remediation decisions.
Misconception diagnosis & remediation	Wrong answers are classified into misconception types (conceptual gap, calculation error, terminology confusion, etc.) and answered with a targeted re-teaching strategy, not a repeated explanation.
Multilingual delivery	Lessons, questions, and narration are produced natively in English, Hindi, Tamil, Spanish, or Hinglish.
Natural voice narration	Every lesson can be narrated using Microsoft Edge neural text-to-speech voices matched to the selected language.
Animated teacher avatar	A lightweight, expressive avatar (happy, thinking, explaining, questioning, encouraging, concerned) reacts live to the teaching flow.
Downloadable lesson videos	Complete lessons can be rendered into an MP4 video combining narration, a talking avatar, and subject-aware visuals (graphs, diagrams, flowcharts, equations, timelines).
Subject-aware visuals	The Visual Engine automatically chooses appropriate visual types per subject — equations and graphs for physics/maths, flowcharts and code panels for programming, timelines and maps for history/geography.
Learning path recommendations	The Learning Path Engine recommends the next concept or topic to study, and can generate 7-day study plans and revision plans.
Persistent learner memory	All progress, assessment history, misconceptions, and teaching events are stored in a local SQLite database and survive across sessions.

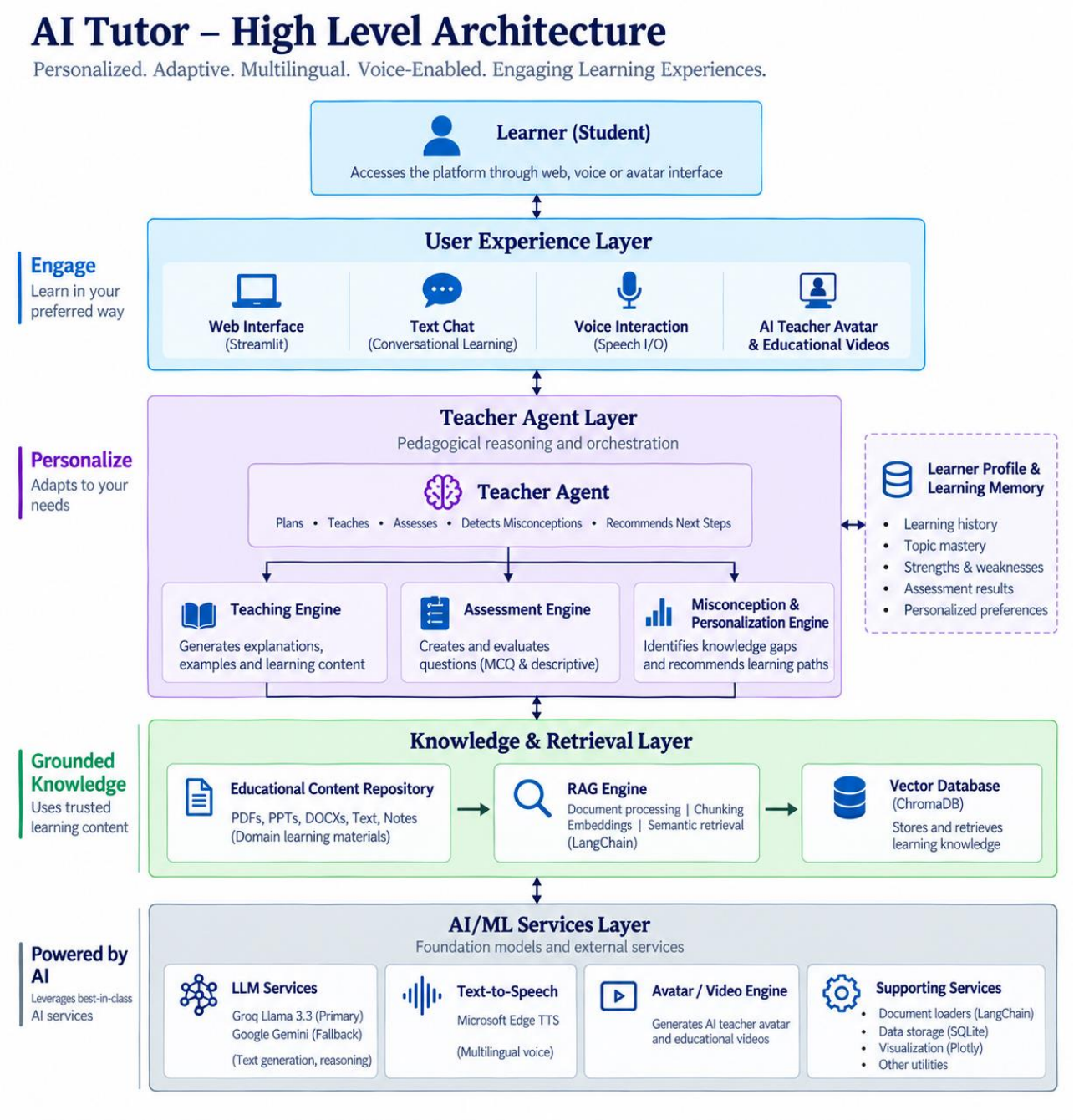
Feature	Description
Resilient LLM provider layer	A provider-agnostic LLM factory automatically prefers the free, high-quota Groq API and falls back to Gemini, so a rate limit on one provider doesn't stop a lesson midway.

4. System Architecture

PedagogyEngine AI follows a layered, modular architecture. Streamlit provides the presentation layer; a set of independent "engine" modules provide the pedagogical and generative logic; a thin persistence layer stores learner state; and an LLM provider abstraction decouples the whole system from any single AI vendor.

4.1 High-Level Flow

The diagram below traces a single learning session end to end: uploaded material is ingested and grounded, the Teacher Agent uses the learner's profile and memory to decide what to teach, that decision becomes multimodal lesson content, and the learner's response feeds back into assessment, diagnosis, and long-term mastery tracking.



4.2 Core Modules

Module	Layer	Responsibility
app.py	Streamlit UI & orchestration	Page routing, session-state management, sidebar API-key entry, wires every engine together, renders chat/progress/video components.
config.py	Central configuration	App constants: learner levels, goals, teaching styles, depth levels, supported languages, teacher actions, mastery thresholds, RAG/audio/video settings, and validation helpers.

Module	Layer	Responsibility
llm_provider.py	LLM factory / abstraction	Provider-agnostic get_chat_llm(): tries Groq first, then Gemini, so every engine is decoupled from a specific vendor SDK.
rag_engine.py	Knowledge grounding (RAG)	Loads PDF/TXT/DOCX/PPTX, chunks text, embeds with a local HuggingFace model, stores/retrieves via ChromaDB, and builds source-cited grounded context.
lesson_generator.py	Lesson content generation	Turns a topic + retrieved context + learner profile into a structured, Pydantic-validated lesson plan (modules, explanations, visuals, interactive questions).
teacher_agent.py	Pedagogical decision engine	Implements the Understand → Plan → Explain → Demonstrate → Question → Evaluate → Adapt → Continue loop; decides the next teaching action as structured JSON.
assessment_engine.py	Answer evaluation	Grades MCQ and free-form answers, computes correctness/confidence/mastery deltas, with an LLM-graded path and a deterministic fallback.
misconception_engine.py	Error diagnosis & remediation	Classifies the misconception behind a wrong answer and generates a targeted re-teaching strategy and follow-up question.
learner_profile.py	Learner state manager	Tracks per-topic and per-concept mastery, strong/weak/developing concepts, active misconceptions, and builds the personalization context fed to every prompt.
learning_memory.py	Session/event memory	Records teaching actions, lesson lifecycle events, assessment and remediation cycles, and builds recent-memory context for continuity between sessions.
learning_path.py	Curriculum sequencing	Recommends the next concept/topic, and builds 7-day study plans and revision plans from the learner's mastery data.
visual_engine.py	Visual planning	Selects subject-appropriate visual types (equation, graph, diagram, flowchart, timeline, map, code, table, etc.) and builds a visual specification for each teaching moment.
audio_engine.py	Text-to-speech	Generates multilingual narration audio using Microsoft Edge neural voices, with per-language voice mapping and adjustable speaking rate.
avatar_component.py	On-screen avatar	Renders a lightweight HTML/CSS/emoji-driven animated teacher avatar in the Streamlit UI, reacting with expressions tied to the current teaching action.
frame_renderer.py	Video frame rasterization	Uses Pillow and Matplotlib to draw individual video frames: an animated talking-avatar face plus a subject-aware visual panel and caption bar.
video_engine.py	Video assembly	Plans lesson scenes and timelines, and — where FFmpeg is available — renders per-module clips and concatenates them into a final narrated MP4 lesson video.
database.py	Persistence layer	SQLite schema and access layer for learner profiles, topic/concept progress, assessment history, and learning events.

Module	Layer	Responsibility
components/	Reusable UI widgets	chat.py, progress.py and video_player.py implement the chat interface, progress dashboards, and in-app video playback inside Streamlit.

5. AI/ML Models Used

The system deliberately uses a small set of well-understood, low-cost or free-tier models rather than a single monolithic model, so that reasoning, retrieval, and speech are each handled by the model best suited to that job.

Purpose	Model	Why it was chosen
Reasoning / lesson generation LLM (primary)	Groq — Llama 3.3 70B Versatile (llama-3.3-70b-versatile)	Chosen as the default provider because Groq offers a free, high-quota tier with very low latency inference, which keeps a multi-step teaching session (planning → lesson → questions → grading → remediation) responsive.
Reasoning / lesson generation LLM (fallback)	Google Gemini — gemini-2.5-flash (configurable via GEMINI_MODEL)	Used automatically whenever a Groq key is not configured or a Groq call fails, so existing Gemini-only users keep working unchanged.
Embedding model (RAG)	sentence-transformers/all-MiniLM-L6-v2 (via HuggingFaceEmbeddings, CPU)	A small, fast, locally-run sentence embedding model used to vectorize uploaded document chunks and learner queries for semantic retrieval — no external embedding API call is made.
Text-to-speech model	Microsoft Edge neural voices (via the edge-tts library) — e.g. en-US-ChristopherNeural, hi-IN-SwaraNeural, ta-IN-PallaviNeural, es-ES-AlvaroNeural	Neural, natural-sounding voices selected per supported language for lesson narration.

6. RAG Implementation

Retrieval-Augmented Generation is what keeps the tutor grounded in the learner's own uploaded material instead of hallucinating from general model knowledge. It is implemented end-to-end in rag_engine.py.

6.1 Ingestion Pipeline

- **1. Document loading —** PDF (PyPDFLoader), TXT (TextLoader), and DOCX (Docx2txtLoader) use LangChain community loaders. PPTX files use a custom PPTXLoader built on python-pptx that extracts slide titles, text boxes, bullet points, table cells, and speaker notes, preserving the slide number as metadata.
- **2. Metadata normalization —** every loaded chunk is tagged with filename, file type, and a human-readable location (e.g. "Page 4" or "Slide 7") so later citations can point back to the exact source.
- **3. Text cleaning —** loaded text is normalized (whitespace, encoding artifacts) before splitting.

- **4. Chunking —** a RecursiveCharacterTextSplitter breaks documents into chunks of 700 characters with a 120-character overlap (both configurable via RAG_CHUNK_SIZE / RAG_CHUNK_OVERLAP), balancing retrieval precision against context continuity.
- **5. Embedding —** each chunk is embedded locally using sentence-transformers/all-MiniLM-L6-v2 on CPU — no embedding API call or extra cost.
- **6. Storage —** embeddings and metadata are persisted in a local Chroma vector store (data/chroma_db), under the collection educational_material.

6.2 Retrieval & Grounding

- **Similarity search —** retrieve() runs a top-k similarity search (default k = 5, configurable via RAG_TOP_K) against the vector store for the current concept/topic/query, with optional metadata filtering.
- **Scored retrieval —** retrieve_with_scores() additionally returns a similarity score per chunk, used where the calling engine needs a confidence signal.
- **Grounded context construction —** build_grounded_context() wraps every retrieved chunk in explicit [SOURCE n: filename — location] / [END SOURCE n] boundaries before it is inserted into a prompt, so the LLM can clearly distinguish retrieved evidence from its own general knowledge. When nothing relevant is found, the engine inserts an explicit NO_RELEVANT_SOURCE_MATERIAL_FOUND marker with an instruction not to fabricate content attributed to the material.
- **Source references for the UI —** get_source_references() de-duplicates and returns clean (filename, location, file type) tuples so the Streamlit UI can show the learner exactly which page/slide an explanation came from.
- **Health checks —** health_check() and get_collection_info() report whether the vector store is reachable and how many chunks are indexed, so the app can warn the learner if no material has been uploaded yet.

7. Personalization Approach

Personalization is treated as first-class state, not a one-off setting. It is captured once via a learner profile setup screen and then threaded through every subsequent prompt and decision in the system.

7.1 Profile Dimensions

Dimension	Options	Effect on teaching
Level	Beginner · Intermediate · Advanced	Sets the assumed prior knowledge and vocabulary complexity of every explanation.
Learning Goal	Concept Understanding · Exam Preparation · Interview Preparation · Practical Application · Project Learning · Revision	Shapes lesson emphasis — e.g. exam prep favors practice questions, interview prep favors crisp definitions and trick-question style probing.
Teaching Style	Visual · Step-by-Step · Example-Based · Socratic · Story-Based · Exam-Focused	Changes how the same concept is delivered — e.g. Socratic style leads with guiding questions rather than direct explanation.

Dimension	Options	Effect on teaching
Depth	Concise · Standard · Detailed · Deep Dive	Controls explanation length and how many supporting examples/derivations are included.
Language	English · Hinglish · Hindi · Tamil · Spanish	Lesson text, questions, and narration are generated natively in this language, not translated after the fact.
Session Duration	5 / 10 / 20 / 30 / 45 / 60 minutes	Mapped to a teaching mode (Quick Concept → Deep Dive) that controls how much content and how many practice cycles fit in the session.

7.2 Adaptive, Mastery-Aware Personalization

Beyond the static profile, LearnerProfileManager continuously tracks dynamic learner state that further personalizes teaching in real time: per-topic and per-concept mastery scores recalculated after every assessment, lists of strong, weak, and developing concepts, and any active (unresolved) misconceptions, which bias the Teacher Agent toward remediation. This combined state — explicit preferences plus continuously-updated mastery — means two learners with identical settings but different quiz performance will receive measurably different lessons on the same topic.

8. Assessment Methodology

assessment_engine.py implements a two-track evaluation system that mirrors how a human tutor grades — objectively for closed-form questions, and with judgment for open-ended ones — feeding a single, comparable mastery signal back into the learner profile either way.

8.1 Question Types

Six question types are supported end-to-end: MCQ, SHORT_ANSWER, CONCEPTUAL, PROBLEM_SOLVING, APPLICATION, and OWN_WORDS.

8.2 MCQ Evaluation

evaluate_mcq() resolves the learner's selected option against the correct option index (robust to the option being given as a letter, index, or the option text itself), and returns a deterministic correct/incorrect result with full confidence — no LLM call is required for this path, keeping grading instant and free.

8.3 Free-Form Evaluation

evaluate_free_form() is used for short-answer, conceptual, problem-solving, application, and "own words" questions. The learner's answer, the expected answer/rubric, and the question context are sent to the LLM with instructions to return correctness, a confidence score, and a mastery delta as strict JSON. If the LLM call fails or returns unparseable output, _fallback_free_form_evaluation() applies a deterministic keyword/heuristic comparison so grading never blocks the lesson.

8.4 Mastery Calculation

calculate_mastery() converts a stream of assessment results into a single, bounded mastery score (0–1) per concept, which is then compared against the thresholds below to decide the next teaching action and to update the progress state shown in the learner's dashboard (NOT_STARTED → IN_PROGRESS → STRUGGLING / IMPROVING → MASTERED).

Setting	Value	Purpose
Initial mastery threshold	0.70	Baseline bar a concept must clear to be considered understood on first pass.
Strong mastery threshold	0.85	Above this, the Teacher Agent is guided toward INCREASE_DIFFICULTY / CONTINUE rather than further practice.
Weak mastery threshold	0.50	Below this, the agent is guided toward SIMPLIFY / REEXPLAIN and the Misconception Engine is engaged.
Max remediation attempts	2	Caps how many times the same concept is re-taught before the strategy is changed rather than repeated.
Final assessment length	5 questions	Number of questions used for a concluding, summative check of a topic.

9. Multilingual Implementation

Multilingual support is implemented as a native generation feature, not a post-hoc translation layer — the LLM is instructed to produce the lesson, questions, and explanations directly in the target language, which produces far more natural phrasing (especially for Hinglish, which has no single "correct" translation) than translating English output afterward.

9.1 Supported Languages & Voice Mapping

Language	Default Edge TTS Voice	Notes
English	en-US-ChristopherNeural	Default; used for all technical/global subject matter.
Hinglish	hi-IN-MadhurNeural	Mixed Hindi-English register, common among Indian learners studying technical subjects.
Hindi	hi-IN-SwaraNeural	Full Hindi explanation and narration.
Tamil	ta-IN-PallaviNeural	Full Tamil explanation and narration.
Spanish	es-ES-AlvaroNeural	Full Spanish explanation and narration.

9.2 How It Flows Through the System

- The learner's chosen language is stored as part of the personalization profile and threaded into every prompt sent to LessonGeneratorEngine, TeacherAgent, AssessmentEngine, and MisconceptionEngine.

- A LANGUAGE_ALIASES normalization map (in both config.py and audio_engine.py) tolerates variant inputs — e.g. "hi", "Hindi", "hindi english" — and resolves them to one canonical language key used everywhere else in the system.
- The same canonical language key selects: the language the LLM is instructed to write in, the Edge TTS voice used for narration, and the on-screen avatar's language-aware supported set.
- Because grounding (RAG) and language are independent concerns, a learner can upload English-language source material and still be taught in Tamil or Hinglish — the retrieved context is passed to the LLM, which explains it in the target language.

10. Voice Implementation

audio_engine.py provides text-to-speech using Microsoft Edge's neural voices via the open-source edge-tts library — no paid TTS API key is required.

10.1 Capabilities

- **Per-language default voices —** each supported language maps to a specific, natural-sounding neural voice (see Section 9.1 for the full language-to-voice mapping); a custom voice can be substituted per language via set_voice().
- **Adjustable speaking rate —** four presets are supported (slow: -20%, normal: +0%, fast: +15%, very_fast: +30%), mapped through normalize_rate().
- **Text cleaning —** clean_text() strips markdown/formatting artifacts from generated lesson text before it is sent to the TTS engine, so narration doesn't read out asterisks or headers.
- **Duration estimation —** estimate_duration() approximates spoken length from word/character count, used to plan video scene timing before audio is actually generated.
- **Async generation —** generate_speech() wraps edge_tts.Communicate in a managed event loop (_run_async) so it can be safely called from within Streamlit's synchronous execution model.
- **Lesson-level narration —** generate_lesson_audio() takes an entire lesson plan and produces per-module narration audio files (data/audio/), ready to be played back live in the UI or stitched into a video.

11. Avatar / Video Generation Approach

Rather than depending on a third-party avatar/video-generation API (e.g. a commercial talking-head service), PedagogyEngine AI renders its own lightweight 2D avatar and video pipeline entirely in-house using open-source libraries. This keeps the feature completely free to run and removes any external service dependency or per-minute rendering cost.

11.1 Live In-App Avatar

avatar_component.py renders an animated teacher avatar directly inside the Streamlit UI using HTML/CSS/emoji-based markup (no video rendering needed for this live view). The avatar supports seven expressions — neutral, happy, thinking, explaining, questioning, encouraging, concerned — each mapped to an emoji and styling, and

switches expression live as the Teacher Agent moves through its teaching actions (e.g. "explaining" during EXPLAIN, "questioning" during ASK, "encouraging" after a correct answer).

11.2 Downloadable Lesson Video

For a persistent, shareable artifact, the system can render a complete narrated lesson as an MP4 file, built from two cooperating modules:

- **frame_renderer.py —** draws the actual pixel frames using Pillow (for compositing) and Matplotlib (for charts/graphs), independent of Streamlit. Each frame combines: (1) a stylized cartoon avatar face with alternating mouth-open/mouth-closed states to create a lightweight "talking" animation synced to the narration audio, and (2) a subject-aware visual panel driven by the same visual_data produced by the Visual Engine — so the video shows the same graph, diagram, equation, or flowchart that appears in the live lesson, not a generic placeholder — composed with a caption/title bar.
- **video_engine.py —** plans the lesson into scenes (intro, explanation, demonstration, visual, question, evaluation, remediation, recap, outro) with calculated durations, builds a scene timeline, and — when the FFmpeg binary is detected on the system (shutil.which("ffmpeg")) — renders each lesson module into a short clip by combining generated frames with narration audio (probing exact audio duration via ffprobe), then concatenates all module clips into one final MP4 using FFmpeg's concat demuxer.

12. APIs and Third-Party Services

The system is intentionally built so that only one service — the LLM — requires a paid or externally-hosted API in the general case, and even that has a free tier. Every other capability (embeddings, vector search, TTS, video rendering) runs locally.

Service	Used For	Credentials Required	Type
Groq API	Primary LLM inference (lesson generation, teacher decisions, grading, misconception diagnosis)	API key (free tier, no card required) via GROQ_API_KEY / Streamlit secrets	Cloud API
Google Gemini API	Fallback LLM inference when Groq is unavailable	API key via GEMINI_API_KEY / Streamlit secrets	Cloud API
Hugging Face sentence-transformers	Local embedding model for RAG (all-MiniLM-L6-v2)	None — model runs locally on CPU	Local (offline after first download)
ChromaDB	Local vector database for storing/retrieving document embeddings	None — embedded, file-based store under data/chroma_db	Local
Microsoft Edge TTS (edge-tts)	Multilingual neural text-to-speech narration	None — uses Microsoft Edge's public read-aloud service via the open-source edge-tts client	Free web service (unofficial API)

Service	Used For	Credentials Required	Type
FFmpeg	Audio/video probing and MP4 assembly for downloadable lesson videos	None — local system binary	Local binary
LangChain (core / community / text-splitters)	Document loading, text splitting, and vector-store/embedding integration glue	None — library only	Local library
pypdf, docx2txt, python-pptx	Parsing uploaded PDF, DOCX, and PPTX study material during RAG ingestion (rag_engine.py)	None	Local library
Streamlit	Web application framework and hosting UI	None (or a Streamlit Community Cloud account for hosted deployment)	Framework / hosting
Pillow, Matplotlib, Plotly	Video frame rasterization (Pillow, Matplotlib) and in-app charts/progress visualizations (Plotly)	None	Local library